



MINISTÉRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DO PIAUÍ – UFPI
CAMPUS SENADOR HELVÍDIO NUNES DE BARROS – PICOS
BACHARELADO EM SISTEMAS DE INFORMAÇÃO



Sistema de Monitoria Acadêmica (SIGMON)

Implementação do Docker

DISCENTES:

Allana Camilly de Sousa Martins
Gabriel dos Santos Silva
Janielle de Jesus Barbosa
José Henrique dos Santos Reis
Lucas Gonçalves de Jesus Pereira



OBJETIVO

O objetivo deste trabalho foi desenvolver uma aplicação web utilizando Python e Flask, capaz de realizar operações em um banco de dados MongoDB.

Para facilitar a implantação e execução da aplicação, utilizamos Docker Compose, criando um ambiente composto por múltiplos containers.

Além disso, utilizamos um banco de dados NoSQL distribuído em Replica Set, aumentando a disponibilidade e a confiabilidade do sistema.

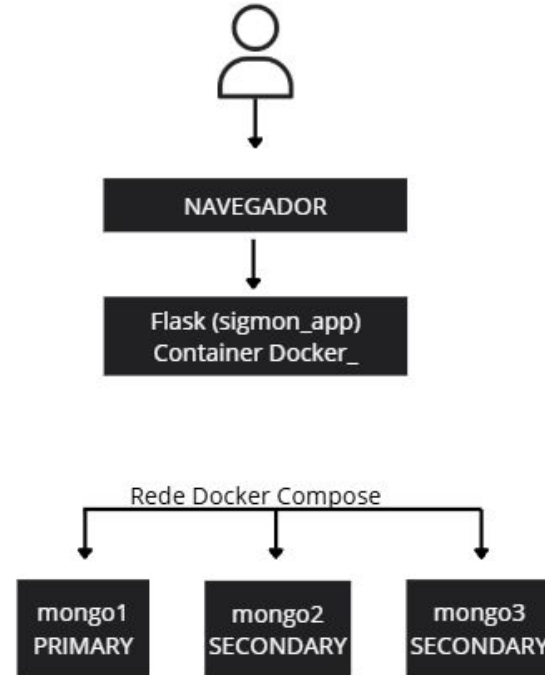


TECNOLOGIAS

- Python como linguagem principal;
- Framework Flask para desenvolvimento da aplicação web;
- MongoDB como banco de dados NoSQL;
- Docker para containerização;
- Docker Compose para gerenciamento dos containers.

ARQUITETURA DA APLICAÇÃO

1. Usuário acessa pelo navegador.
2. A requisição chega ao servidor Flask.
3. O Flask processa a lógica do sistema.
4. As informações são armazenadas no MongoDB Replica Set.
5. Os dados retornam ao usuário.



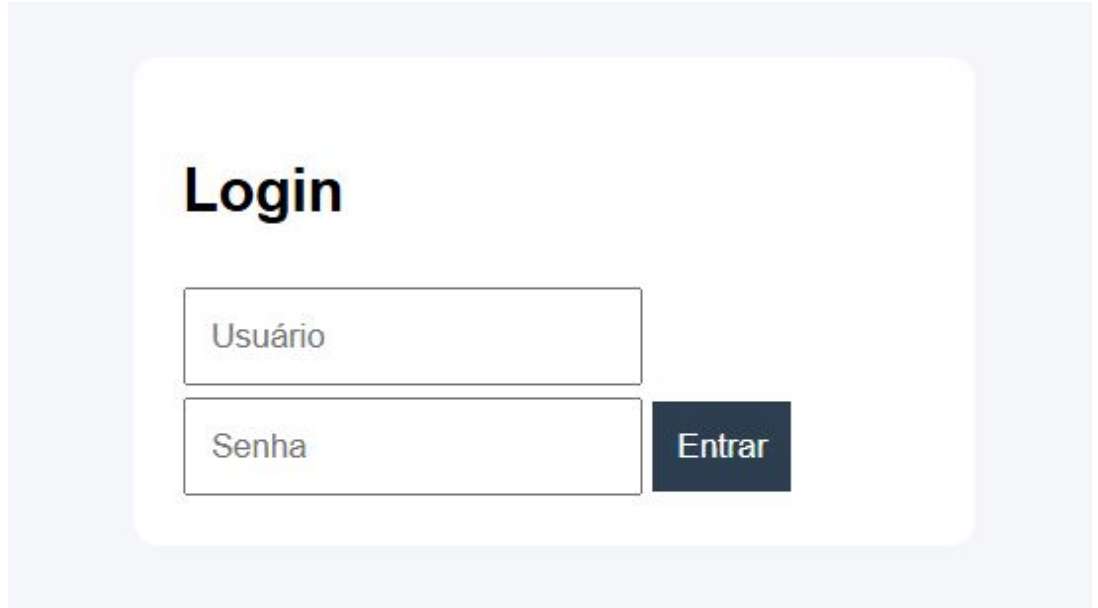


FUNCIONALIDADES DA APLICAÇÃO

- Sistema de monitoria
 - Plataforma para facilitar o atendimento dos alunos pelos monitores.
 - Permite realizar o cadastro de monitores a alunos.
 - Registra as dúvidas dos alunos sobre determinados assuntos.

FUNCIONALIDADES DA APLICAÇÃO

- Tela 1:
 - Permite que o usuário realize o login.

A mockup of a login screen is shown within a light blue rounded rectangle. The screen has a white background. At the top, the word 'Login' is displayed in a large, bold, black font. Below it are two white input fields with thin black borders. The first field is labeled 'Usuário' in a light gray font. The second field is labeled 'Senha' in a light gray font. To the right of the 'Senha' field is a dark blue button with the word 'Entrar' in white text.



FUNCIONALIDADES DA APLICAÇÃO

- Tela 2:
 - Solicita os atendimento.
 - Lista de atendimentos.

Sistema de Monitoria UFPI

[Dashboard](#) [Alunos](#) [Monitores](#) [Atendimentos](#) [Login](#)

Atendimentos

Tipo de Monitoria:

Disciplina Desejada:

Descrição / Dúvida:

Solicitar Atendimento

Lista de Atendimentos

Tipo	Disciplina	Monitor Responsável	Descrição/Dúvida
Presencial	Sistemas operacionais	A definir (Sem monitor ativo)	dúvida sobre escalonamento



FUNCIONALIDADES DA APLICAÇÃO

- Tela 3:
 - Cadastra os monitores e suas disciplinas.

The screenshot displays the 'Sistema de Monitoria UFPI' web application. The top navigation bar is dark blue with the title 'Sistema de Monitoria UFPI' and links for 'Dashboard', 'Alunos', 'Monitores', 'Atendimentos', and 'Login'. The main content area is titled 'Monitores' with a person icon. Below this is a form to register a monitor, consisting of five input fields: 'Nome do monitor', 'Disciplina que é monitor', 'Curso', 'Período atual (Ex: 5)', and 'Docente da disciplina', followed by a 'Cadastrar' button. At the bottom, there is a section titled 'Lista de Monitores' containing a table with the following headers: 'Nome', 'Disciplina', 'Curso', 'Período', and 'Docente'.

Nome	Disciplina	Curso	Período	Docente
------	------------	-------	---------	---------



FUNCIONALIDADES DA APLICAÇÃO

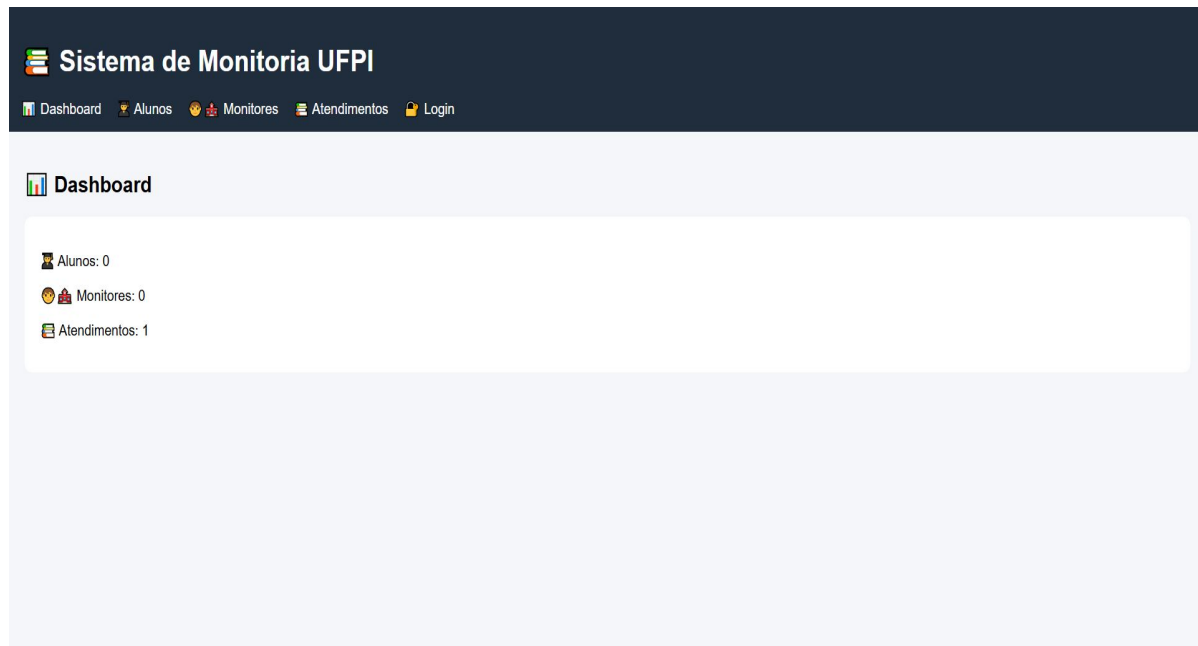
- Tela 4:
 - Cadastra os alunos com seus dados.

The screenshot shows the 'Sistema de Monitoria UFPI' interface. At the top, there is a dark blue header with the system name and a navigation bar containing links for 'Dashboard', 'Alunos', 'Monitores', 'Atendimentos', and 'Login'. Below the header, the 'Alunos' section is active, indicated by a person icon and the text 'Alunos'. This section contains a form for registering a student with five input fields: 'Nome do aluno', 'E-mail', 'Curso', 'Período (Ex: 3)', and 'Disciplinas (Ex: SO, Redes)'. A dark blue 'Cadastrar' button is positioned to the right of the last field. Below the form is a section titled 'Lista de Alunos' which contains a table with the following headers: 'Nome', 'E-mail', 'Curso', 'Período', and 'Disciplinas'. The table body is currently empty.

Nome	E-mail	Curso	Período	Disciplinas
------	--------	-------	---------	-------------

FUNCIONALIDADES DA APLICAÇÃO

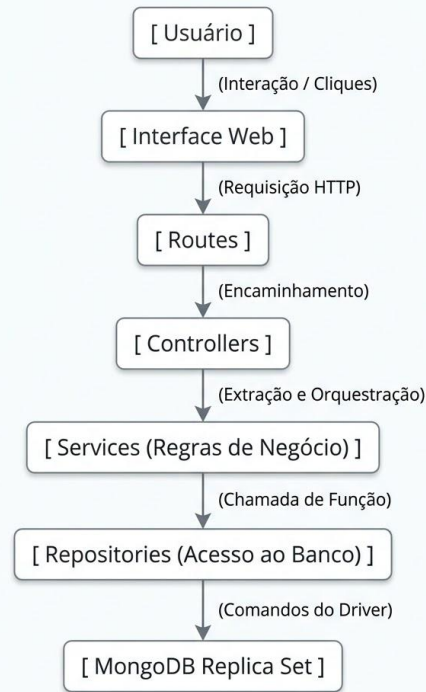
- Tela 5:
 - Dashboard para monitorar todos os dados registrados



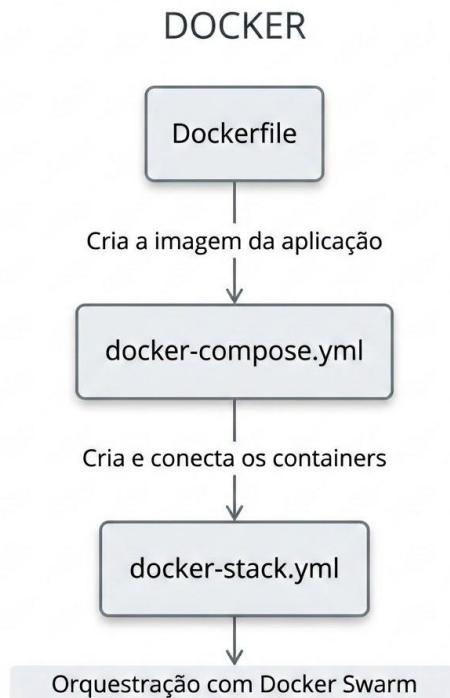
COMUNICAÇÃO DA APLICAÇÃO COM O BANCO DE DADOS

Fluxo da comunicação

- O usuário realiza uma ação pela interface da aplicação.
- A requisição chega às **Routes**, que fazem o roteamento das URLs.
- Os **Controllers** recebem a requisição, extraem os dados do usuário e repassam o fluxo.
- Os **Services** processam a solicitação e aplicam todas as regras de negócio.
- Os **Repositories** isolam os comandos do driver e acessam o MongoDB.
- Os dados são persistidos no Replica Set e a resposta retorna ao usuário.



ESTRUTURA DOCKER DA APLICAÇÃO





DOCKERFILE

- Define a imagem da aplicação.
- Instala o Python e as dependências.
- Copia os arquivos do projeto.
- Inicia a aplicação Flask.

```
FROM python:3.13.7-slim

WORKDIR /app

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

Configuração do Dockerfile da aplicação, evidenciando a definição da imagem base em Python, a instalação das dependências do sistema e o comando de inicialização do servidor Flask.



ORQUESTRAÇÃO COM DOCKER COMPOSE

- Cria os containers da aplicação e do MongoDB.
- Configura a rede entre os containers.
- Define volumes, portas e variáveis de ambiente.
- Facilita a execução local com um único comando.

```
> docker-compose.yml
version: "3.9"

Run All Services
services:
  Run Service
  app:
    image: sigmon_app:latest
    ports:
      - "5000:5000"
    environment:
      MONGO_URI: mongodb://mongo1:27017,mongo2:27017,mongo3:27017/?replicaSet=rs0
      DATABASE_NAME: sigmon
      SECRET_KEY: sigmon_ufpi_2026
    depends_on:
      - mongo1
      - mongo2
      - mongo3
  networks:
    - sigmon_network
```

Configuração do serviço da aplicação, destacando a injeção da URI de conexão com o cluster do banco e o isolamento na rede virtual interna.

ORQUESTRAÇÃO COM DOCKER COMPOSE

- Cria os containers da aplicação e do MongoDB.
- Configura a rede entre os containers.
- Define volumes, portas e variáveis de ambiente.
- Facilita a execução local com um único comando.

```
>Run Service
mongo1:
  image: mongo:8.0
  command: ["--replSet", "rs0", "--bind_ip_all"]
  ports:
    - "27017:27017"
  volumes:
    - mongo1_data:/data/db
  networks:
    - sigmon_network
```

Configuração do nó de banco de dados, evidenciando o comando de inicialização do Replica Set e o mapeamento de volume físico para persistência dos dados.



ESCALABILIDADE COM DOCKER SWARM

- O que é o Docker Swarm?
 - Ferramenta nativa de orquestração de contêineres do Docker.
 - Permite agrupar várias máquinas em um único cluster.
 - Contribui para a alta disponibilidade e tolerância a falhas da aplicação.

ESCALABILIDADE COM DOCKER SWARM

- Aplicando no nosso projeto:
 - **Réplicas:** O Swarm garante que sempre existam 2 instâncias do seu app rodando simultaneamente (Alta Disponibilidade).
 - **Restart Policy:** Se um contêiner falhar, o Swarm o reinicia automaticamente para manter o estado desejado.

```
deploy:
  replicas: 2
  restart_policy:
    condition: on-failure
```



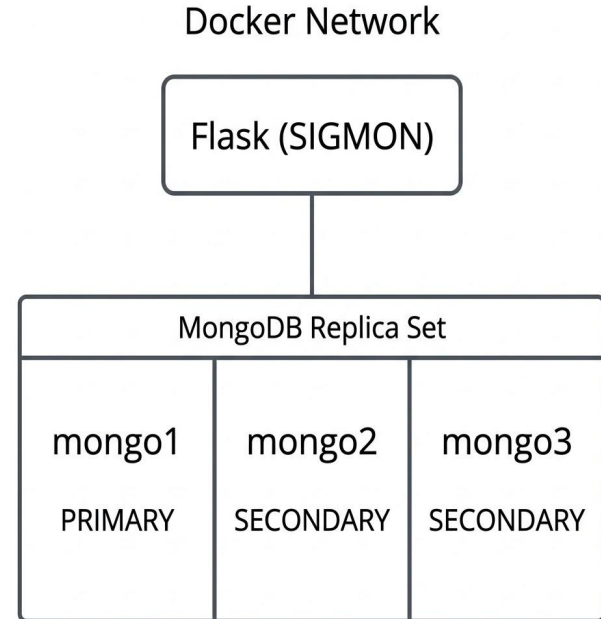
ESCALABILIDADE COM DOCKER SWARM

- Trabalhando com mais de uma máquina:
 - **Driver Overlay:** Cria uma rede virtual que interconecta todos os nós do Swarm.
 - O **Driver Overlay** cria uma rede virtual distribuída, permitindo que o `app.py` localize e se comunique com os serviços do MongoDB através dos nós do cluster.

```
networks:  
  sigmon_network:  
    driver: overlay
```

ARQUITETURA DOS CONTAINERS

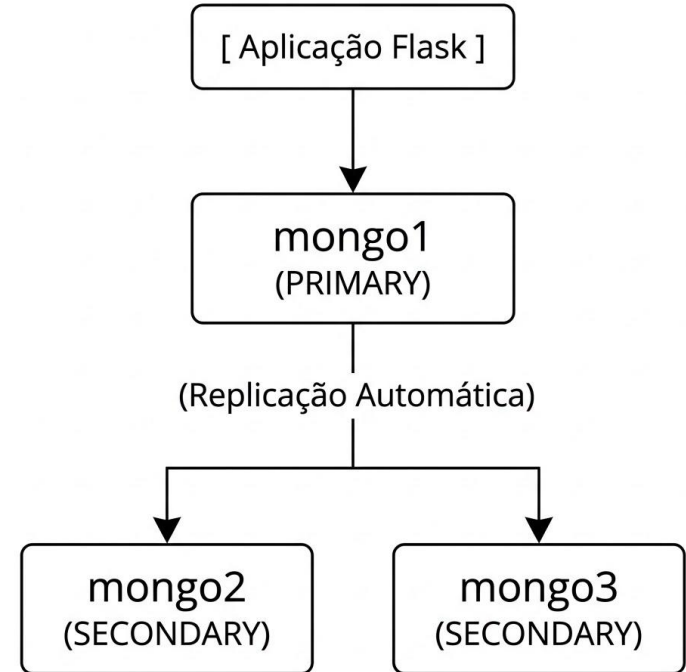
- Os 4 containers operam de forma isolada dentro de uma rede privada criada automaticamente pelo **Docker Compose**.
- A aplicação localiza o banco de dados utilizando os próprios nomes dos containers (mongo1, mongo2, mongo3), sem depender de IPs estáticos ou localhost.
- Essa estrutura garante comunicação direta entre os processos do sistema, mantendo o banco de dados protegido de acessos externos.





MONGODB REPLICA SET

- Três instâncias do banco operam em conjunto para garantir a **disponibilidade** contínua do sistema.
- Os cadastros do banco principal são espelhados para os secundários, gerando **redundância** de dados em tempo real.
- Se o servidor principal cair, um secundário assume o controle automaticamente, assegurando a **tolerância a falhas**.
- A inicialização do cluster e a definição dos nós são 100% automatizadas pelo script **init-replica.js**.
- Conexão unificada da aplicação com o cluster:
`mongodb://mongo1:27017,mongo2:27017,mongo3:27017/?replicaSet=rs0`





MONGODB REPLICA SET

- Três instâncias do banco operam em conjunto para garantir a **disponibilidade** contínua do sistema.
- Os cadastros do banco principal são espelhados para os secundários, gerando **redundância** de dados em tempo real.
- Se o servidor principal cair, um secundário assume o controle automaticamente, assegurando a **tolerância a falhas**.
- A inicialização do cluster e a definição dos nós são 100% automatizadas pelo script **init-replica.js**.
- Conexão unificada da aplicação com o cluster:
`mongodb://mongo1:27017,mongo2:27017,mongo3:27017/?replicaSet=rs0`

```
r > JS init-replica.js > ...
rs.initiate({
  _id: "rs0",
  members: [
    {
      _id: 0,
      host: "mongo1:27017"
    },
    {
      _id: 1,
      host: "mongo2:27017"
    },
    {
      _id: 2,
      host: "mongo3:27017"
    }
  ]
})
```

OBRIGADO PELA ATENÇÃO!